SESIÓN 6: Comunicación UDP

Práctica con solución

Cliente.java y Servidor.java

En los sockets UDP no se establece conexión, por lo que el esquema cliente-servidor no está tan bien definido como cuando se utiliza el protocolo TCP. Podemos considerar al servidor como a la máquina que espera un mensaje y responde, y al cliente la máquina que inicia la comunicación.

Tanto uno como otro si desean ponerse en contacto necesitan saber en qué ordenador y en qué puerto están escuchando el uno al otro. El esquema general de comunicación sigue los siguientes pasos:

1. El servidor crea un socket asociado a un puerto local para escuchar peticiones de clientes. Permanece a la espera de recibir peticiones.
2. El cliente creará un socket para comunicarse con el servidor. Para enviar datagramas necesita conocer su IP y el puerto por el que escucha. Utilizará el método *send()* del socket para enviar la petición en forma de datagrama.
3. El servidor recibe las peticiones mediante el método *receive()*del socket. En el datagrama va incluido además del mensaje, el puerto y la IP del cliente emisor de la petición; lo que le permite al servidor conocer la dirección del emisor del datagrama. Utilizando el método *send()* del socket puede enviar la respuesta al cliente emisor.
4. El cliente recibe la respuesta del servidor mediante el método *receive()* del socket.
5. El servidor permanece a la espera de recibir más peticiones.

Vamos a realizar una práctica en Java mediante el envío de mensajes UDP en la cual un Servidor sea capaz de contar el número de letras ‘a’ que recibe de un Cliente. Para ello, necesitaremos un Cliente que leerá una cadena de texto de la entrada estándar, la envíe al servidor y espere una respuesta del mismo.

Vayamos por partes, empecemos por el comportamiento del Servidor, que es la parte de la comunicación que se encargará de procesar la información que le haga llegar el Cliente.

SERVIDOR.JAVA

El programa servidor comienza definiendo un paquete de bytes que será el espacio de memoria donde se almacenará el mensaje que le envíe el cliente:

```
//Esperando Datagrama
System.out.println("Servidor esperando datagrama...");
DatagramPacket recibo;

byte[] bufer = new byte[1024];
recibo = new DatagramPacket(bufer, bufer.length);
```

A continuación, asocio ese espacio de memoria a un socket, y en concreto a uno definido en el puerto 12345. En ese momento, el software quedará esperando a recibir información en ese puerto:

```
//Asocio el socket al puerto 12345 y me quedo esperando
DatagramSocket socket = new DatagramSocket(12345);
socket.receive(recibo);
```

En cuanto llegue información en ese puerto, se ejecutarán las siguientes instrucciones: accedemos al contenido del paquete con el método *"getData()"*, le quitamos los espacios del inicio y del final con *"trim()"* y el resultado lo mostramos por pantalla:

```
String mensaje = new String(recibo.getData()).trim();
System.out.println("Servidor recibe: "+mensaje);
```

A continuación, procesamos los datos de entrada, que en este caso, consiste en contar el número de 'a's que tiene el dato recibido:

```
//Contar el numero de letras a
int contador = 0;
for (int i=0; i < mensaje.length(); i++) {
    if (mensaje.charAt(i) == 'a')
        contador++;
}
```

A través del "DatagramPacket" que definimos al inicio, somos capaces de extraer la dirección IP local y el puerto:

```
//Direccion origen del mensaje
InetAddress IPOrigen = recibo.getAddress();
int puerto = recibo.getPort();
```

Estos datos los utilizaremos para enviar la respuesta al Cliente. Para ello, definiremos de nuevo un paquete de bytes en el cual utilizaremos el primer byte para el contador de caracteres (de hecho, lo castearemos de *int* a *byte*). Este paquete lo asociaremos al socket de antes y lo enviaremos con *send()*.

```
//Enviando datagrama al cliente
System.out.println("Enviando numero de apariciones de la letra 'a': "+contador);
byte b = (byte)contador;
byte[] enviados = new byte[1024];
enviados[0] = b;

DatagramPacket envio = new DatagramPacket(enviados, enviados.length, IPOrigen, puerto);
socket.send(envio);
```

Por último cerramos el socket:

```
//Cerrar socket
System.out.println("Cerrando conexion...");
socket.close();
```

CLIENTE.JAVA

Desde el lado Cliente, comenzamos definiendo los datos del Servidor al que enviar el mensaje. En este caso, y para realizar la práctica con más sencillez, el Servidor estará en la misma máquina, por lo que extraeremos la dirección IP local mediante el método “getLocalHost” de la clase “InetAddress”.

```
//Datos del Servidor al que enviar mensaje
InetAddress IPServidor = InetAddress.getLocalHost();
int puerto = 12345;
```

A continuación leemos la frase que queremos que analice el servidor y lo pasamos a bytes. Esto se hace así porque la información se transmite siempre al nivel más bajo del modelo OSI, que es el nivel físico y cuya unidad de transmisión son los bits (que se agrupan a su vez en bytes).

```
//Introducir datos por teclado
System.out.println("Introduce mensaje: ");
String cadena = sc.nextLine();

byte[] enviados = new byte[1024];
enviados = cadena.getBytes(); //convertir cadena a bytes
```

Ahora que sabemos dónde queremos enviar la información y, sobre todo, cual es el mensaje a transmitir, podemos montar un datagrama mediante la clase “DatagramPacket”:

```
//Creamos el datagrama que irá al servidor
DatagramPacket envio = new DatagramPacket(enviados, enviados.length,
                                          IPServidor, puerto);
```

Y también le podemos asociar un socket, en este caso, en el puerto 34567 y realizar el envío, que lo llevamos a cabo mediante el método “send()” de la clase “DatagramSocket”:

```
//Realizamos el envio
DatagramSocket clientSocket = new DatagramSocket(34567);
clientSocket.send(envio);
```

En el momento que se realiza el envío, llega la parte más importante de la gestión del Cliente y es mantener **una espera activa** hasta que el Servidor termine de procesar la información.

Esta espera activa la conseguimos creando un datagrama, lo asociamos con el socket que creamos con anterioridad y llamamos al método receive(). En ese punto, el Cliente se queda esperando a recibir el datagrama del Servidor:

```
//Recibiendo datagrama del servidor
byte[] recibidos = new byte[2];
DatagramPacket recibo = new DatagramPacket(recibidos, recibidos.length);
System.out.println("Esperando datagrama...");
clientSocket.receive(recibo);
```

Cuando el Cliente recibe el datagrama del Servidor, es momento de analizar el contenido del mismo, lo cual vamos a conseguir con el método “getData” de la clase “DatagramPacket”.

En este “DatagramPacket” vamos a tener un vector de 1024 bytes, pero como ya sabemos que solo vamos a ocupar el primero de ellos, accedemos a la posición 0 de la estructura de datos:

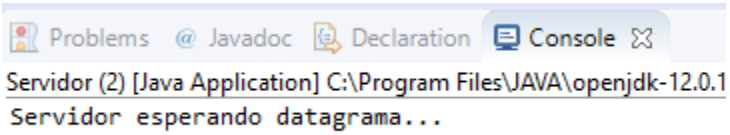
```
//Obtener el numero de caracteres
byte[] vector_bytes = recibo.getData();
int numero = vector_bytes[0];
System.out.println("Recibo numero de caracteres 'a' que son: "+numero);

clientSocket.close();
```

Finalmente, y cuando imprimimos el mensaje, cerramos el socket con el método “close()”.

EJECUCIÓN DE SERVIDOR Y CLIENTE

Un ejemplo de ejecución sería el siguiente. Arrancamos el Servidor desde Eclipse…



Y desde el bash/cmd, arrancamos el Cliente que nos pide que introduzcamos el texto a analizar. Introducimos, por ejemplo, “Hola Caracola” y nos quedamos en el momento de “espera activa”. Por su parte, el Servidor recibe la cadena, la procesa, devuelve el resultado y el Cliente acaba imprimiendo el resultado por pantalla.

El Cliente se queda así:

```
C:\Users\franp\eclipse-workspace\COM32\src>java Cliente
Introduce mensaje:
Hola caracola
Esperando datagrama...
Recibo numero de caracteres 'a' que son: 4
```

El Servidor finaliza su ejecución y se queda así:

